LECTURE-2

History of programming -

- Machine language (0/1)
  - Prone to error
  - Tedious
  - Scalable ☒
- Assembly level language
  - Uses english alphabets
  - Tightly coupled
  - Prone to error
  - less scalable
- Procedural Programming.
  - introduced everything except OOPS
    - Eg functions, loops, blocks etc.
  - less able in complex problems
  - set of instruction
    - Eg C Language
- Object-Oriented programming Language
  - Real-world modelling
  - Data Security
  - Scalable/Reusable

⇒ In real world we interact with "OBJECTS"

CLASS- Class is a blueprint of the application

OBJECT- object is an instance of the class.
objects have 1) characteristics
2) Behaviour

Eg.

```
Class Car {
    string brand;
    string model;
    bool isEngineOn;
    void start { }
    void stop { }
}

Car BMW = new Car();
```

Pillars of OOPS -

- → Abstraction
- → Encapsulation
- → Inheritance
- → Polymorphism

ABSTRACTION

→ When we drives a car we accelerate, break, gear shift but we don't know how it internally works or we don't know mechanism of engine or break. there should be an interface (car body) for the driver to use these properties.

→ Same with phone we don't know mechanism we only deal with phone screen to deal with the mechanism or use it.

→ In abstraction we hide the implementation of the structure and show only essential or necess. things to user.

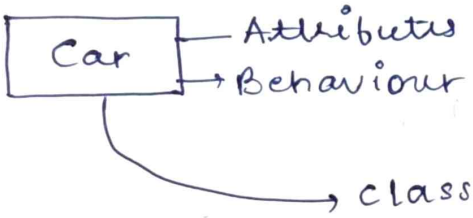
<u>Virtual functions-</u> Virtual functions are those functions that are ~~defined~~ declared but not ~~declared~~ defined. It can be defined by child class who extends it.

```
class Car {
  public:
    virtual void startEngine() = 0
    virtual void shiftGear(int gear) = 0;
  };
```

Here Car is a interface.

ENCAPSULATION

→ Encapsulation is the process in which we bind the attributes & methods. Like a medicine capsule.




```
class Car{
    // variables → Attributes
    // Functions → Behaviour
}
```

→ Encapsulation also helps in Data security.

→ Access Modifiers - It helps Encapsulation to set the access limit of Attributes & Behaviour.

Types -
* Public
* Private
* Protected

→ Public can be accessed from anywhere

→ Private can be accessed only in the declaring or its own class

→ Protected can be accessed from its own and child class only.

```
Class Car {
  ~~public~~ private:
      string brand;
      string model;
      bool isEngineOn;
      int currentspeed;
  ~~private~~ public:
      SportsCar ( ) {

      }

      void startEngine ( ) {

      }

  }
```

Here we private attributes just because we cannot manipulate it without permission.

We have to set setter and getter functions or behaviour to take access or manipulate private member.

```
int getspeed ( ) {
    return oldspeed;
}

void setspeed (int newspeed) {
    oldspeed = newspeed;
}
```